Typographics Art: everything in one file This file contains the README and the main documents of the open-source project Typographics Art (https://typographics-art.org, https://github.com/typ0graphic5/tg-art). It is meant to be handed to an AI assistant as a whole. Good first questions: What is Typographics Art? How is it different from ASCII art? Can a machine still read a work? How big do the PDFs get, and why? --- <!-- README.md --> ## Overview --- ## Ask your AI what this is Everything in this repository, README and documents, in one file: **about/typographics-art.md** (44 000 characters, also at typographics-art.org/llms-full.txt). Or hand it the same text as a one-pager, made with the technique itself: **about/typographics-art.pdf**. Then ask: *What is Typographics Art, and can you still read this page?* ## The idea **ASCII art** builds a picture out of characters, but the characters mean nothing. A person sees a face; a machine sees `@@%#*+=-:.`. The text is gone. **Typographics Art** (TG Art) builds a picture out of a *real text*. Every character of the text is set once, in its original order. Only its typography changes: size, weight, width and colour. A person sees the picture. A language model, a search engine or a screen reader reads the text, exactly as it was written. | | ASCII art | Typographics Art | |---|---|---| | What is chosen | the **character** for each cell of a grid | the **typography** for each character of a text | | The text | lost, replaced by shading characters | kept 1:1, in reading order | | What a machine reads | `@@%%##**++==--::..` | *"Wer reitet so spät durch Nacht und Wind? ..."* | | Fonts | monospace, one size | variable fonts: weight, width, size, colour per glyph | | Format | plain text | HTML, SVG, PDF with a text layer | ### What a person sees, and what an AI reads The portrait of Frederick Douglass above is set from his own *Narrative* (1845), chapters V–VIII. Open [`gallery/douglass.svg`](gallery/douglass.svg), select all and copy, or hand the file to any model: ```text As to my own treatment while I lived on Colonel Lloyd's plantation, it was very similar to that of the other slave children. I was not old enough to work in the field, and there being little else than field work to do, I had a great deal of leisure time. ... ``` That is the whole point: **the artwork is also the document.** No alt text needed; the content *is* the alt text. ## Readable for AI, measured We do not just claim it. Every number below is measured and reproducible ([method](docs/ai-readability.md)). | How a machine gets the text | Result | |---|---| | **HTML or SVG**: reading the characters (DOM, XML) | **100 % exact.** Checked by the test suite on every commit. | | **PDF text layer** in stored order (`pdftotext -raw`) | **100 % exact** in all 6 tested settings | | **PDF**, tools that rebuild the visual layout (`pdftotext` default) | exact at legibility *high*; 97–99 % of the word order kept at *medium* and *free* | | **Only the picture**: OCR (Tesseract 5.5) on a PNG at 2× | legibility *high*: **98.9–99.6 %** of words · *medium*: 88.7–92.7 % · *free*: 68.8–82.3 % | | **Only the picture**: a vision-language model (Claude) reading crops | **100 %** of 217 words, at *high*, *medium* and *free* (first sample) | So: **give machines the HTML or SVG**, and they get the text perfectly, at any artistic setting. PDF works the same when read in stored order, and with every tool at legibility *high*. If only an image travels (a screenshot, a social post), choose legibility *high* and the text survives even pure image recognition. A first sample with a vision-language model read every word, even at *free*, where classic OCR stumbles. More models and larger samples: [help is welcome](CONTRIBUTING.md). ## Demonstrator: Moby-Dick in one picture The strongest proof is a whole book. [**typographics-art.org/demo/moby-dick**](https://typographics-art.org/demo/moby-dick/) sets the complete text of *Moby-Dick* (1.2 million characters, 135 chapters) as a single 2 × 3 m poster in three panels, one motif per part of the book. Next to the zoomable poster, the page shows the proof: - **Complete:** the text read back from the artwork has the same SHA-256 hash as the source text. - **Read back correctly:** from the SVG master and from the print PDF, 100 random passages are found at their original positions, chapter structure and word frequencies are identical. - **Usable by AI:** a defined question set (narrator, ship, captain, ...) has every answer grounded in the extracted text; OCR samples measure what survives in a mere photo of the poster. - **Reproducible:** download the book from Project Gutenberg, run two scripts, compare the hashes. Full report: demo/moby-dick/proof.md. Click anywhere on the poster and the page shows the text at that spot, read from the artwork's own line offsets. ## One-pagers: a whole document on one page `cli/onepage.mjs` puts the complete text of a multi-page PDF (or any text file) on a single page. The letters are tiny; together they form the document's title in large type, or, with `--motif`, a picture above the title. A person sees at a glance what the page is. A model reads every word from the PDF's text layer: the type size does not matter to it. | | | | |---|---|---| |
Swiss Federal Constitution
102 pages, 228 576 characters |
EU AI Act
144 pages, 595 706 characters |
AI instructions
a code review agent's rules | Every example is read back with MuPDF and compared with its source; all of them match. Ten examples (laws, declarations, PEP 8, AI agent instructions, this repository's AGENTS.md) are in examples/. ```sh node cli/onepage.mjs document.pdf --title "Short title" -o onepage.pdf --png preview.jpg node cli/onepage.mjs document.pdf --title "Short title" --motif photo.jpg -o onepage.pdf ``` **File size.** Every glyph carries its own weight, width, size and colour, and a PDF embeds a font for every weight × width × size it uses. Measured and optimised in docs/file-size.md: outlines rounded, empty accessibility tags left out, one type size by default (which also makes the picture sharper). Now three of the five PDF examples are *smaller* than their source (US Constitution 1.6 MB from 3.8 MB, EU AI Act 2.2 MB from 2.5 MB), the others up to 2.8 times larger; short plain-text sources grow most (3 KB of instructions → 67–105 KB). `--weights 2` makes a one-pager smaller still. The [examples table](examples/README.md#file-size-honestly) lists every size; the original motif images are in examples/motifs/. Share the PDF, not a picture of it: at this size, a plain image is grey texture to OCR and vision models. And keep it honest: the visible title must name the document, and tiny text must never hide instructions from the people who see the page. ## Hidden messages A work can carry a message that only the holder of a key can read. The work itself does not change: it holds only its public text, nothing invisible. The key records, for every character of the message, one position in that text where the same character stands, chosen at random among all its occurrences (a book cipher). With the key and the right work, the message comes back; with any other work, the key's text fingerprint does not match. ```sh node cli/tg-secret.mjs hide work.pdf --secret "Meet at the harbour, 6 pm." # prints the key: tg1.... node cli/tg-secret.mjs reveal work.pdf --key tg1.... # prints the message ``` Works for PDF (read with MuPDF), SVG, HTML and text; the web generator has the same under *Hidden message*. Keep the key like a password: with key and work, anyone can read the message. Characters that do not occur in the public text cannot be hidden; the tool names them. ## The three rules 1. **Fidelity.** The text is never changed, reordered or shortened. Filler text (for example Lorem ipsum for a finer picture) is allowed only if it is declared. 2. **Text stays text.** The work is published as real text: HTML, SVG, or PDF with a text layer. Extracting it yields exactly the source. 3. **Only typography varies.** No shapes, backgrounds or images besides the glyphs. Full definition: docs/technique.md. ## How it works ```mermaid flowchart LR T[Text] --> M[Measure every glyph
in every weight and width] P[Photo] --> D[Tone map
ink demand per area] M --> S[Set glyph by glyph
choose weight, width, size
error diffusion] D --> S S --> C[Colour each glyph
ink + paper = photo colour] C --> O[HTML · SVG · PDF
text stays text] ``` 1. **Measure.** For each character: advance width and ink coverage at ten weights (and three widths for fonts with a width axis). 2. **Map.** The photo becomes an "ink demand" per area: darkness on light paper, brightness on dark paper. 3. **Set.** Line by line, each glyph gets the weight, width and size whose ink best matches the area it covers. The leftover error is passed on to the next glyph and the next line (error diffusion), so tones average out correctly. 4. **Fit.** A binary search finds the font size at which the whole text fills the picture exactly. 5. **Colour.** In colour mode each glyph's ink is chosen so that ink and visible paper average to the photo colour. Details and formulas: docs/algorithm.md. ## Use it ### In the browser Go to **typographics-art.org/app.html**, paste a text, drop a photo. Presets: *AI-safe* (always legible), *Portrait*, *Poster*. Export HTML, SVG, PNG, or print to PDF. Everything runs locally; nothing is uploaded. ### As a library No build step, no dependencies. ES modules for browsers and Node ≥ 20. ```js import { typeset, toPng } from './src/browser.js'; import { toSvg, toStandaloneHtml, isFaithful } from './src/index.js'; const sheet = await typeset(text, imageElement, { font: 'roboto', // the one typeface: Roboto Flex (weights 100–1000) legibility: 'medium', // high | medium | free color: 'photo', // photo | gray | mono paper: 'light', // light | dark unit: 'char', // char | word (one style per word: calmer to read) stretch: true, // also vary letter width (fonts with a width axis) shape: true, // follow edges within each letter width: 960, }); isFaithful(sheet); // true: the set text equals the source toStandaloneHtml(sheet); // HTML snippet, text as characters toSvg(sheet); // SVG, text as characters toPng(sheet); // PNG data URL (image only) ``` The pure modules (`layout`, `tone`, `color`, `render`, `typeset`) run anywhere; `browser.js` measures real fonts and reads pixels. All options and defaults: src/options.js. ### On the command line Needs Node 22+ and Chrome or Chromium (`CHROME_PATH` if it is not found); PDFs also need MuPDF (`mutool`), which checks every printed strip and gives the page its exact size, and get smaller with qpdf. Writes PDFs with a real text layer. ```sh node cli/tg-art.mjs photo.jpg --text poem.txt --fill 12000 -o poem.pdf node cli/tg-art.mjs photo.jpg --string "Hello world" --paper dark -o hello.svg node cli/tg-art.mjs --help ``` The output format follows the file extension: `.svg`, `.html`, `.pdf` (text stays text), `.png`, `.jpg` (image only). ## Gallery | | | | |---|---|---| |
Learning to Read · Frederick Douglass |
My Inventions · Nikola Tesla |
The Wanderer · Nietzsche · C. D. Friedrich | |
The Great Wave · Melville · Hokusai |
La Gioconda · Walter Pater · Leonardo |
The House of Usher · Edgar Allan Poe | |
Fuji-no-Yama · Lafcadio Hearn · Hokusai |
The Star · H. G. Wells · Hubble |
Earthrise · Genesis · Apollo 8 | All fifteen works, each switchable to its original image: typographics-art.org/gallery. Every image is in the public domain ([sources](gallery/photos/README.md)); every text is a contiguous passage of a public-domain work, without filler. **Yours?** [Submit an artwork](https://github.com/typ0graphic5/tg-art/issues/new?template=artwork.yml). Each preview links to the SVG, which is the work itself: open it and search the text. All works are reproducible from [`gallery/works.config.json`](gallery/works.config.json) with `npm run gallery`. ## Where this is useful - **Art that explains itself.** A portrait made of the sitter's biography, a city made of its history. Any reader, human or machine, gets both. - **Accessible images.** The content is the description: screen readers read the text, no separate alt text is needed. - **Findable art.** Search engines and AI crawlers index the words, not just a file name. - **Provenance.** The text inside a work can carry a statement, a licence or a signature that survives copy and paste. - **Datasets and posters.** Poems, manifestos, lyrics and code as pictures that stay quotable. ## FAQ **Is this a calligram?** Calligrams (like Apollinaire's) arrange text along a shape, usually by hand. Typographics Art keeps a normal reading layout and creates the picture purely through per-glyph typography, computed from any photo. **Does Lorem ipsum filler break the fidelity rule?** No, as long as it is declared. The app and the CLI report exactly how many characters are filler, and the filler always comes after your text. **Why variable fonts?** A weight axis (and sometimes a width axis) gives each glyph a continuous range of ink, so tones can be matched precisely without changing the letter. **Can I use it commercially?** Yes. The code is MIT-licensed; the fonts are under the SIL Open Font License. Mind the rights to the texts and photos you use. ## Contributing Typographics Art is meant to become a shared technique, like ASCII art. Ideas, fonts, better algorithms, translations, benchmarks and artworks are all welcome. Start with CONTRIBUTING.md; please follow the [code of conduct](CODE_OF_CONDUCT.md). ```sh npm test # node --test, no dependencies npm run dev # app at http://localhost:8080 npm run gallery # re-render the gallery (needs Chrome/Chromium) ``` Writing about Typographics Art? The [press kit](docs/press-kit.md) has texts, facts and images. ## Licence [MIT](LICENSE) © The Typographics Art contributors --- <!-- docs/technique.md --> ## Definition of the technique # Typographics Art: definition **Typographics Art** (short **TG Art**) is the technique of forming a picture from a given text by setting that text character for character, in its original order, and varying only the typographic properties of each character. It is the counterpart of ASCII art. ASCII art chooses *which character* fills each cell of a grid. Typographics Art keeps the characters the text dictates and chooses *how each one is set*. ## Rules A work is Typographics Art when it follows all three rules. ### 1. Fidelity - Every character of the source text appears exactly once, in reading order. - No character is replaced, inserted, reordered or dropped. - Whitespace may be normalised: runs of spaces, tabs and line breaks become a single space, and line breaks may fall at any space. - A word longer than a line may be broken between two characters. No hyphen is added. - **Filler** (for example Lorem ipsum to reach a higher resolution) is allowed only when it is declared: it follows the source text, and the work states how many characters are filler. ### 2. Text stays text - The work is published in a format that keeps the characters as text: HTML, SVG, or a PDF with a text layer. - Extracting the text, by copy and paste, a screen reader or a program, yields the source text (rule 1 applies after whitespace normalisation). - A raster export (PNG, JPEG) is a reproduction of the work, not the work itself. ### 3. Only typography varies Per character, a work may vary: - font size, - font weight and other axes of a variable font (width, optical size, slant), - typeface, - colour and opacity, - horizontal spacing (letter and word spacing). It may not add shapes, strokes, backgrounds behind individual characters, or images. A single uniform paper colour is allowed. ## Legibility The technique does not require every character to be legible to the eye at viewing size. Very fine works (tens of thousands of characters) are read by zooming in or by machines. Implementations should offer a legibility setting that limits how thin, small or faint a character may become. ## Hidden messages A work may be paired with a key that selects characters of its public text in a secret order (a book cipher, `src/secret.js`). This does not touch the rules above: the work still contains exactly its declared text and nothing invisible; the message lives only in the key. Tiny or invisible text must never be used to hide anything in the work itself. ## Naming - Technique: *Typographics Art*, short *TG Art*. - A work: *a Typographics Art work*, *a TG Art piece*. - German: *Typographics Art*, short *TG-Art*. --- <!-- docs/algorithm.md --> ## How the setting works # How the reference implementation works The pipeline lives in `src/`. Everything except `browser.js` is pure and runs in Node. ## 1. Glyph metrics (`browser.js`) For every distinct character of the text, at ten weight steps (and, for fonts with a width axis, at 75 %, 100 % and 125 % width) of the chosen variable font: - **advance**: the width the browser lays the glyph out with, measured in the DOM at 40 px; - **coverage**: the share of the glyph's cell (advance × line height) covered by ink, measured by rendering it to a canvas and summing alpha; - **ink profile**: the share of ink in the left/right and top/bottom halves of the cell. The optical size axis is pinned (`'opsz' 40`), so advances scale linearly with font size. `typeset()` loads the font's stylesheet first if the page does not link it; otherwise the browser would silently measure a fallback font. ## 2. Tone map (`tone.js`) The image is centre-cropped to the output aspect ratio and reduced to at most 720 px wide. Each pixel becomes an *ink demand* between 0 and 1 (darkness on light paper, brightness on dark paper). Contrast, brightness and an unsharp mask ("emphasise edges") are applied. Summed-area tables make the mean demand and mean colour of any rectangle an O(1) lookup. ## 3. Candidates and lookup tables (`layout.js`) A legibility level allows a set of weights, size factors and (in grey mode) lightening steps. Each combination is a candidate with an effective coverage ``` eff = coverage(char, weight) × size × (1 − grey) × lineRatio / 0.92 ``` For every character, a table maps (target tone, size of the previous glyph) to the cheapest candidate: ``` cost = (eff − target)² + 0.00035·(1 − size) + 0.0002·grey + 0.004·|size − previousSize| ``` A further term `0.0012·|stretch − 1|` lets width change only where it buys tone. The size term keeps neighbouring glyphs at similar sizes; without it, small glyphs next to full-size ones read like stray capitals. ## 4. Setting (`layoutAt`) Lines are filled word by word. For each glyph, the target is the mapped tone of the area it will cover, plus half of the previous glyph's error, plus the error the line above left in that column (error diffusion). With `shape` on, the target is the tone of the four half-cells weighted by the glyph's ink profile, so a glyph on an edge takes the tone of the side its ink is on. A word is set tentatively; if it does not fit, it moves to the next line and is set again there. In word mode, one candidate is chosen for the whole word from its average coverage. ## 5. Fitting (`fit`) A binary search finds the largest line height at which the whole text fits the area, then rounds it down to a multiple of 1/64 px (stepping further down in the rare case where the rounded height does not fit, because glyph widths follow the chosen weights). If the text does not fit even at 3 px, the sheet grows downwards instead of dropping characters. ## 6. Colour (`color.js`) In photo-colour mode, each glyph's ink is chosen so that ink and visible paper average to the photo colour of its cell: ``` ink = paper + (target − paper) / coverage ``` If that leaves the RGB cube, the vector towards the paper is shortened as a whole, which keeps the hue. A legibility cap keeps ink colours apart from the paper. ## 7. Output (`render.js`) Lines are justified by distributing the remaining width over their inner spaces. That moves every glyph after the first space to the right, at the end of a line by a median 1.5 em and up to 5 % of the width (measured on the Swiss constitution one-pager). So once a line's words are fixed, its glyphs are chosen again at their justified positions, twice because the new widths change the spacing slightly, and the line's error is passed on from those positions (`settle` in `src/layout.js`). Without this, vertical edges frayed by several letters and colours slipped at the ends of lines. A repeated character starts at least 0.12 of its own size after the previous one (`REPEAT_GAP`). PDF extractors treat a glyph that repeats the previous character closer than 0.1 em as a "fake bold" duplicate and drop it (MuPDF, `stext-device.c`); a tiny *l* followed by a large *l* lost a letter that way. HTML uses one span per run of equally styled glyphs; SVG uses `tspan`s that are positioned explicitly after every space. Both keep the characters in reading order, which is what makes rule 2 of the technique hold. ## PDF at physical size, always one page The CLI prints the sheet as HTML in headless Chrome (`Page.printToPDF`) at its physical size. Chrome embeds the variable-font instances as Type 3 fonts and keeps every glyph as text. Chrome prints a few hundred thousand glyphs per page in about a minute, but did not finish a 1.2-million-glyph page within 50 minutes. Large sheets are therefore printed in strips of at most about 220 000 glyphs and assembled onto **one page** by `cli/pdf-assemble.js` (a MuPDF script): each strip becomes a Form XObject at its offset, vectors, fonts and text layer unchanged, strips in reading order. PDF pages may be up to 5080 mm per side. Details that keep the result exact: - Chrome rounds page sizes (measured up to 0.22 mm off, and 210 mm became 209.89 mm). Each strip is printed 2 mm taller than its content, and the assembly keeps exactly the content height, placed at the exact cumulative offset; every PDF goes through the assembly, so an A4 one-pager is 210 × 297 mm. The last strip reaches down to the sheet's full height even when the text ends a few lines higher. - Every strip is read back with MuPDF before it is used. Chrome (measured with version 154) sometimes leaves single glyphs out of a large print job, depending on the text printed before them in the same job; a strip that lost anything is split in half and printed again, down to single lines. - No content stream may have more than 1 800 000 tokens: pypdf, behind many AI document loaders, stops reading a stream at 2 000 000 and silently drops the rest. Strips above that are split as well (`cli/pdf-tokens.js`). - Line heights are rounded down to a multiple of 1/64 px, the unit browsers store layout positions in, so no renderer has to round them. Checked on a 1 m poster printed once directly and once from four assembled strips: same page size, identical text layer (SHA-256). The images agree within less than a millimetre per line; the remaining difference was measured but not fully explained. Read the text back with MuPDF (`mutool draw -F txt`). Poppler's `pdftotext` keeps only 50 000 characters smaller than 3 points per page at its default resolution (see `docs/ai-readability.md`). Afterwards the PDF is made smaller without visible change: `cli/pdf-slim.js` rounds the Type 3 glyph outlines to whole units of their 1000-unit grid, and qpdf repacks the file. Chrome's accessibility tags are left out: they were one empty `/NonStruct` element per style run. Measurements in `docs/file-size.md`. ## Measuring quality - **Fidelity**: `isFaithful(sheet)` compares the set text with the source. - **Tone fidelity**: Pearson correlation between target tone and achieved coverage over all glyphs (`toneFidelity`). Error diffusion lowers it locally on purpose, so treat it as a rough indicator. --- <!-- docs/ai-readability.md --> ## Machine readability, measured # AI readability: method and results Typographics Art claims that its works stay readable for machines. This page shows how that was measured, so anyone can repeat it and improve the numbers. Measured on 2026-10-17 with the reference implementation (v0.1.0), Chromium headless, Tesseract 5.5.3 (`eng`), Poppler `pdftotext` 26.08. ## Setup - **Text:** Shakespeare's Sonnet 18 (620 characters), filled with declared Lorem ipsum to 3 000 characters. The filler contains words like *officia*, which matters for ligatures (see below). - **Image:** a red disc above a dark green band on a light background, 640 × 800 px. - **Settings:** legibility *high*, *medium*, *free* × unit *char* (per letter) and *word* (per word); Roboto Flex, photo colours, light paper, width 900 px, width axis and shape awareness on. - **Reference:** the text in the SVG output (all `<text>` content, whitespace normalised). The test suite separately asserts that this equals the source. ## What was measured | Channel | Tool | Metric | |---|---|---| | HTML / SVG | DOM `innerText`, XML text content | exact equality with the source | | PDF, stored order | `pdftotext -raw` | exact equality with the SVG text | | PDF, layout reconstruction | `pdftotext` (default) | exact equality; word-order similarity | | Image only | `tesseract` on a PNG rendered at 2× | share of source words recovered in order (word accuracy), character similarity | Word accuracy is the number of words in the longest common subsequences between source and OCR output (Python `difflib`), divided by the number of source words. ## Results | Legibility · unit | HTML/SVG | PDF raw | PDF layout | OCR words | OCR characters | |---|---|---|---|---|---| | high · char | exact | exact | exact | 98.9 % | 99.8 % | | high · word | exact | exact | exact | 99.6 % | 99.9 % | | medium · char | exact | exact | 97.0 % word order | 88.7 % | 96.9 % | | medium · word | exact | exact | 97.3 % word order | 92.7 % | 98.0 % | | free · char | exact | exact | 97.9 % word order | 82.3 % | 94.1 % | | free · word | exact | exact | 99.1 % word order | 68.8 % | 72.8 % | ## Vision-language model (first sample) Classic OCR knows only letter shapes. A vision-language model also knows language, so it should cope better with strong size and weight changes. - **Model:** Claude (Anthropic, Opus 5.5), reading images directly. - **Material:** one crop of five full lines per legibility level (*high*, *medium*, *free*; letter mode), cut from the 2× PNG, taken from the Lorem ipsum part so the word order cannot be known in advance. Medium and free were re-rendered with a different Lorem sequence after it turned out that the first medium crop repeated words already seen in the high crop; that reading was discarded. - **Procedure:** transcription written down first, then compared with the ground truth from the SVG. | Legibility | Words | Correct | |---|---|---| | high | 58 | 100 % | | medium | 65 | 100 % | | free | 94 | 100 % | This is a small sample with one model, on crops rather than whole pages; whole pages at screen resolution are harder. It does show the direction: where Tesseract drops to 82 % at *free*, the language-aware model read everything. ## Findings that changed the code Measuring found two real defects, both fixed before these results: 1. **Ligatures dropped letters in PDFs.** Chrome set "ffi" in *officia* as one ligature glyph, and its PDF text layer then mapped that glyph to "ff": the extracted text read *offcia*. Ligatures and kerning are now switched off in every output (`GLYPH_CSS` in `src/render.js`). That also keeps the layout exact, because glyphs are measured one by one. 2. **Word mode lacked size coherence.** Neighbouring words jumped between font sizes, and `pdftotext`'s layout mode moved the very first words of the text elsewhere. Word mode now uses the same size-coherence cost as letter mode. That fixed the start of the text, but over the full text layout-reconstructing extraction still moves some line fragments at medium and free legibility (table above). "Word order" is `difflib` similarity of the word sequences. ## A whole book: the Moby-Dick poster The demonstrator in `demo/moby-dick/` sets 1.2 million characters on 2 × 3 m and runs the same kind of checks at that scale; its report is generated by the build (`demo/moby-dick/proof.md`). Two findings from it: - **PDF extraction depends on the tool.** Poppler's `pdftotext` returns fewer letters than are in the file, mostly in light passages, while MuPDF (`mutool draw -F txt`) returns every visible character. The cause is a deliberate Poppler limit (see below): small glyphs, which light passages consist of, are dropped after the first 50 000 per page. The text is in the file; the proof extracts with MuPDF and reports Poppler separately. - **Invisible direction marks** (U+200E/U+200F around one Hebrew word in the book's etymology) are not carried into the PDF text layer. The SVG master keeps them; the PDF test compares visible characters and says so. - **OCR versus vision models at poster density.** With strong mixing of weights, widths, sizes and light colours, Tesseract recognises only part of the words, while a vision-language model reads the same segment images completely (manual sample in the report). ## Which PDF extractors read a one-pager completely Measured on the ten example one-pagers (`scripts/extractor-check.py`, results in `docs/extractor-check.json`; characters counted, exact text equality checked for MuPDF, which every build does): | Extractor | read completely | notes | |---|---|---| | MuPDF (`mutool draw -F txt`), PyMuPDF | 10 of 10 | exact, in reading order | | pdfminer.six | 10 of 10 | | | pypdf | 10 of 10 | reads at most 2 000 000 tokens per content stream; the print strips are split until every stream stays below that (before: the EU AI Act lost 44 %) | | Poppler `pdftotext` (default 72 dpi) | 6 of 10 | keeps only the first **50 000 characters smaller than 3 × 3 units per page** and drops the rest (`TextOutputDev.cc`, "tiny chars limit"); one-pagers above that lose most of their text (10–30 % read). Its default mode also drops a hyphen that ends a line, as for hyphenated words (1–4 characters on the small examples; `-raw` keeps them) | | Poppler `pdftotext -r 600` | 10 of 10 apart from line-end dashes | the unit shrinks with the resolution, so the limit no longer applies; since one-pagers use one type size (no glyphs below the base size), this holds even for the EU AI Act at 0.5 mm type. With mixed sizes the AI Act reached only 81 % | A known quirk: Chrome draws a character the font lacks (for example "→" in Roboto Flex) from a fallback font, and MuPDF then reads the following space as missing ("→ erlaubt" becomes "→erlaubt"); pdftotext keeps it. The visible characters are identical; the build reports such a page as not exact. The agent library avoids arrows for that reason. Earlier versions of this page blamed Poppler's losses on Chrome's Type 3 fonts; that was wrong. A plain page of 120 000 characters in an ordinary font shows the same cut at 50 000. ## Recommendations - **To hand a work to an AI, use HTML or SVG.** The text is exact at every setting. PDF is exact too when read in stored order; with legibility *high* it is exact for layout-reconstructing tools as well. - **If only an image will travel** (screenshot, social media), use legibility *high*: even pure OCR recovers about 99 % of the words. - **PDF extraction:** prefer MuPDF/PyMuPDF, pdfminer.six or pypdf (table above). With Poppler, pass a high resolution (`pdftotext -r 600` or more) for pages with more than 50 000 small characters. Tools that rebuild the visual layout can move line fragments when sizes vary a lot. ## Not measured yet - More vision-language models, larger samples and whole pages instead of crops. - Other OCR engines, scripts beyond Latin, and very fine works (tens of thousands of characters, where glyphs are a few pixels high). Contributions with reproducible numbers are very welcome; open an issue or pull request. --- <!-- docs/file-size.md --> ## File size, measured # File size: what makes a PDF big, and what can be seen A Typographics Art PDF carries the whole text plus the typography of every letter. This page measures where the bytes go, what was removed without visible change, and which settings change the picture. All numbers come from `scripts/size-study.mjs` (raw data in `docs/file-size-study.json`) on two one-pagers, printed with Chrome 154 and read back exactly in every case. ## Where the bytes are Chrome writes every font instance it prints as a **Type 3 font**: one glyph outline per character used. It makes a separate font for every **weight × width × type size** (measured: 10 weights at 5 sizes gave 100 fonts, at one size 20), and large sheets printed in several strips carry their fonts once per strip. Every change of weight, width, size or colour also starts a new text run on the page. A PDF printed in one piece is therefore, within 3 % on 18 measured pages: ``` Chrome print ≈ 555 B × outlines + 7.2 B × runs + 0.9 B × letters + 14 KB after slimming ≈ 328 B × outlines + 7.4 B × runs + 0.8 B × letters + 7 KB ``` *outlines* are distinct weight × width × size × character, *runs* the style changes (`pdfStats` and `estimatePdfBytes` in `src/render.js`; the app shows this estimate). The picture itself is made of letters only: no image is embedded. ## Removed without visible change | Step | Saving | Why it is safe | |---|---|---| | Round glyph outlines to whole units (`cli/pdf-slim.js`) | −22 % (Swiss constitution) | at most 1/2000 of the type size; at 600 dpi no pixel changed by more than 32 of 255 levels, text identical | | Leave out Chrome's accessibility tags | −12 % (Swiss constitution), −16 % (Basic Law) | they were one empty `/NonStruct` element per style run under one `/Document`: no headings, no paragraphs; text and order are unchanged | | Repack with qpdf | −9 to −80 % (Swiss constitution −9 %, Basic Law −80 %) | lossless (object streams, stronger compression) | ## Which settings can be seen *Likeness* is the correlation between the page and the picture it was set to (title or motif card), both blurred to 1.5 mm, in brightness and in colour (1 = identical). *Change to full* is the plain difference to the version with every variant. All rows include the slimming steps above; from "one width" on, the PDF is untagged. ### Swiss constitution (photo motif, colour) | Setting | PDF | fonts in the PDF | likeness brightness / colour | change to full | |---|---|---|---|---| | all weights, widths and sizes, colour step 6, tagged PDF | 4.53 MB | 282 | 0.940 / 0.924 | 0.00 % | | as published so far (colour step 24) | 3.53 MB | 193 | 0.939 / 0.923 | 0.25 % | | without accessibility tags | 3.12 MB | 193 | 0.939 / 0.923 | 0.25 % | | one width (no condensed/expanded) | 2.75 MB | 100 | 0.938 / 0.924 | 1.16 % | | 4 weights, one width | 1.93 MB | 40 | 0.942 / 0.927 | 1.38 % | | 2 weights, one width | 1.52 MB | 20 | 0.945 / 0.930 | 1.28 % | | 1 weight, one width (a single font) | 1.34 MB | 10 | 0.943 / 0.945 | 3.32 % | | 2 sizes, one width | 1.92 MB | 40 | 0.949 / 0.935 | 1.24 % | | 1 size, one width | 1.65 MB | 20 | 0.963 / 0.954 | 1.46 % | | colour step 48, one width | 2.65 MB | 100 | 0.938 / 0.923 | 1.20 % | | colour step 96, one width | 2.63 MB | 100 | 0.931 / 0.918 | 1.39 % | | 2 weights, 1 size, one width | 1.13 MB | 4 | 0.946 / 0.939 | 1.82 % | | 1 weight, 1 size, one width: only colour or grey is left | 0.85 MB | 2 | 0.911 / 0.900 | 4.03 % | ### Basic Law (title card, grey) | Setting | PDF | fonts in the PDF | likeness brightness / colour | change to full | |---|---|---|---|---| | all weights, widths and sizes, colour step 6, tagged PDF | 0.73 MB | 22 | 0.995 / 0.995 | 0.00 % | | as published so far (colour step 24) | 0.73 MB | 22 | 0.995 / 0.995 | 0.00 % | | without accessibility tags | 0.61 MB | 22 | 0.995 / 0.995 | 0.00 % | | one width (no condensed/expanded) | 0.55 MB | 17 | 0.995 / 0.995 | 0.30 % | | 4 weights, one width | 0.55 MB | 15 | 0.983 / 0.983 | 0.60 % | | 2 weights, one width | 0.58 MB | 12 | 0.977 / 0.977 | 0.77 % | | 1 weight, one width (a single font) | 0.68 MB | 6 | 0.982 / 0.982 | 2.20 % | | 2 sizes, one width | 0.55 MB | 12 | 0.995 / 0.995 | 0.31 % | | 1 size, one width | 0.72 MB | 13 | 0.994 / 0.994 | 0.43 % | | colour step 48, one width | 0.55 MB | 17 | 0.995 / 0.995 | 0.30 % | | colour step 96, one width | 0.55 MB | 17 | 0.995 / 0.995 | 0.30 % | | 2 weights, 1 size, one width | 0.54 MB | 4 | 0.994 / 0.994 | 0.42 % | | 1 weight, 2 sizes, one width | 0.60 MB | 3 | 0.979 / 0.979 | 2.72 % | | 1 weight, 1 size, one width: only colour or grey is left | 0.60 MB | 2 | 0.961 / 0.961 | 5.12 % | What this shows: - **One type size is sharper.** Mixing sizes made horizontal streaks; with one size the photo motif follows its picture more closely (0.963 instead of 0.940) at a third of the size of the full version. On the title card it makes no difference. - **Weight carries the brightness.** Two weights keep the picture; a single weight loses contrast. With neither weights nor sizes left, only colour draws the picture and likeness drops the most. - **Width variants are not worth it.** Dropping condensed and expanded changes nothing measurable and saves 12 %. - **Coarser colours hardly save.** Colour steps of 48 or 96 instead of 24 save 4 %; step 48 looks the same, step 96 worse. Colour changes are cheap in a PDF. One-pagers therefore use **one size and no width variants** by default (`cli/onepage-core.mjs`); `--weights 2` makes them smaller still. In the app, the font-cuts slider limits the weights and shows the expected PDF size. Works made as art (gallery, posters) keep their sizes: there the texture of mixed sizes is a choice. ## Reproduce ```sh node scripts/size-study.mjs # prints every setting, writes docs/file-size-study.json (about 45 minutes) node scripts/size-study.mjs --evaluate # measures the PDFs of the last run again ``` --- <!-- examples/README.md --> ## One-pager examples # Examples: whole documents on one page Each example puts the **complete text** of a document on a single A4 page. The letters are tiny; together they form the document's title in large type, or a picture with the title beneath it ([original motifs](motifs/) for comparison). A person sees at a glance what the page is. A machine reads every word from the PDF's text layer: the type size does not matter to it. Every page is checked by reading it back with MuPDF and comparing it with the source text. The table is generated by `node examples/build.mjs`. > **Important:** the text is read from the PDF, not from the picture. Exported as a plain image (PNG, JPEG), type this small is grey texture to OCR and vision models. Share the PDF. ## Laws | | Document | Source | On one page | File size | Read back from the PDF | |---|---|---|---|---|---| | | **[Bundesverfassung der Schweizerischen Eidgenossenschaft, SR 101, Stand am 3. März 2024](ch-bundesverfassung/onepage.pdf)** · [motif](motifs/ch-bundesverfassung-1874.jpg) | 102 pages, 31,129 words · [original](https://www.fedlex.admin.ch/filestore/fedlex.data.admin.ch/eli/cc/1999/404/20240303/de/pdf-a/fedlex-data-admin-ch-eli-cc-1999-404-20240303-de-pdf-a.pdf) | 228,576 characters at 0.754 mm | 1.65 MB (original 1.05 MB) | ✓ every visible character identical (exact) | | | **[Grundgesetz für die Bundesrepublik Deutschland, gesetze-im-internet.de](de-grundgesetz/onepage.pdf)** | 51 pages, 26,180 words · [original](https://www.gesetze-im-internet.de/gg/GG.pdf) | 190,286 characters at 0.844 mm | 738 KB (original 266 KB) | ✓ every visible character identical (exact) | | | **[The Constitution of the United States of America, House Document 110-50 (pocket edition, with the Declaration of Independence)](us-constitution/onepage.pdf)** · [motif](motifs/us-constitution-hintermeister-1925.jpg) | 85 pages, 44,527 words · [original](https://www.govinfo.gov/content/pkg/CDOC-110hdoc50/pdf/CDOC-110hdoc50.pdf) | 303,928 characters at 0.661 mm | 1.63 MB (original 3.83 MB) | ✓ every visible character identical (exact) | | | **[Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act), Official Journal](eu-ai-act/onepage.pdf)** · [motif](motifs/eu-ai-act-network.png) | 144 pages, 91,421 words · [original](http://publications.europa.eu/resource/cellar/dc8116a1-3fe6-11ef-865a-01aa75ed71a1.0006.01/DOC_1) | 595,706 characters at 0.446 mm | 2.20 MB (original 2.46 MB) | ✓ every visible character identical | ## Declarations | | Document | Source | On one page | File size | Read back from the PDF | |---|---|---|---|---|---| | | **[Universal Declaration of Human Rights, English text, OHCHR](un-human-rights/onepage.pdf)** | 8 pages, 1,779 words · [original](https://www.ohchr.org/sites/default/files/UDHR/Documents/UDHR_Translations/eng.pdf) | 10,733 characters at 3.617 mm | 137 KB (original 191 KB) | ✓ every visible character identical (exact) | ## AI instructions | | Document | Source | On one page | File size | Read back from the PDF | |---|---|---|---|---|---| | | **[PEP 8 – Style Guide for Python Code (reStructuredText source)](pep-8/onepage.pdf)** | 7,153 words · [original](https://raw.githubusercontent.com/python/peps/main/peps/pep-0008.rst) | 47,423 characters at 1.704 mm | 327 KB (original 50 KB) | ✓ every visible character identical (exact) | | | **[Code review agent: operating instructions (written for this repository)](code-review-agent/onepage.pdf)** | 594 words | 3,514 characters at 6.076 mm | 105 KB (original 3 KB) | ✓ every visible character identical (exact) | | | **[Customer support assistant: policy (fictional shop, written for this repository)](support-assistant/onepage.pdf)** | 557 words | 3,165 characters at 6.628 mm | 84 KB (original 3 KB) | ✓ every visible character identical (exact) | | | **[Research assistant: rules for sources and claims (written for this repository)](research-assistant/onepage.pdf)** | 444 words | 2,625 characters at 7.306 mm | 67 KB (original 3 KB) | ✓ every visible character identical (exact) | | | **[AGENTS.md of this repository](agents-md/onepage.pdf)** | 467 words | 3,002 characters at 6.776 mm | 76 KB (original 3 KB) | ✓ every visible character identical (exact) | ## Agent library | | Document | Source | On one page | File size | Read back from the PDF | |---|---|---|---|---|---| | | **[Wissens- und Aufgabenbibliothek für KI-Agenten: alle 20 Dateien in einem Dokument (written for this repository)](agent-bibliothek/onepage.pdf)** | 30,205 words | 225,554 characters at 0.786 mm | 890 KB (original 224 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Bibliothek: Übersicht, drei Teile, Felder eines Standard-Prompts](agent-library-readme/onepage.pdf)** | 950 words | 7,350 characters at 4.191 mm | 144 KB (original 7 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Bibliothek: Wissen gezielt bereitstellen (neun Bereiche)](agent-library-wissen/onepage.pdf)** | 1,094 words | 8,441 characters at 3.899 mm | 157 KB (original 8 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Regel 1: Quellen und Belege](agent-regel-quellen/onepage.pdf)** | 639 words | 4,432 characters at 5.378 mm | 127 KB (original 4 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Regel 2: Unsicherheit](agent-regel-unsicherheit/onepage.pdf)** | 584 words | 4,022 characters at 5.589 mm | 119 KB (original 4 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Regel 3: Aufgabenbearbeitung](agent-regel-aufgaben/onepage.pdf)** | 567 words | 3,984 characters at 5.698 mm | 122 KB (original 4 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Regel 4: Berechtigungen](agent-regel-berechtigungen/onepage.pdf)** | 521 words | 3,625 characters at 5.937 mm | 116 KB (original 4 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Regel 5: Datenschutz](agent-regel-datenschutz/onepage.pdf)** | 509 words | 3,768 characters at 5.746 mm | 120 KB (original 4 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Regel 6: Qualitätsprüfung](agent-regel-qualitaet/onepage.pdf)** | 537 words | 3,788 characters at 5.742 mm | 118 KB (original 4 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Regel 7: Ausgabeformate](agent-regel-ausgabe/onepage.pdf)** | 553 words | 3,980 characters at 5.616 mm | 122 KB (original 4 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Regel 8: Umgang mit Fremdinhalten](agent-regel-fremdinhalte/onepage.pdf)** | 516 words | 3,835 characters at 5.724 mm | 115 KB (original 4 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Prompts: Softwareentwicklung (7 Prompts)](agent-prompts-code/onepage.pdf)** | 2,572 words | 19,347 characters at 2.64 mm | 207 KB (original 19 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Prompts: Software-Sicherheit (4 Prompts)](agent-prompts-sec/onepage.pdf)** | 1,695 words | 13,532 characters at 3.13 mm | 187 KB (original 13 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Prompts: Recherche (5 Prompts)](agent-prompts-recherche/onepage.pdf)** | 2,548 words | 18,980 characters at 2.674 mm | 200 KB (original 19 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Prompts: Dokumente (5 Prompts)](agent-prompts-dokumente/onepage.pdf)** | 2,460 words | 18,345 characters at 2.712 mm | 201 KB (original 18 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Prompts: Datenanalyse (5 Prompts)](agent-prompts-daten/onepage.pdf)** | 2,181 words | 16,382 characters at 2.888 mm | 188 KB (original 16 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Prompts: Schreiben (5 Prompts)](agent-prompts-schreiben/onepage.pdf)** | 1,966 words | 15,008 characters at 3.001 mm | 185 KB (original 15 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Prompts: Unternehmen (4 Prompts)](agent-prompts-unternehmen/onepage.pdf)** | 2,323 words | 17,639 characters at 2.74 mm | 210 KB (original 18 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Prompts: Projektmanagement (5 Prompts)](agent-prompts-projekt/onepage.pdf)** | 2,490 words | 18,012 characters at 2.743 mm | 196 KB (original 18 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Prompts: Design und Produkt (4 Prompts)](agent-prompts-design/onepage.pdf)** | 2,637 words | 20,051 characters at 2.565 mm | 229 KB (original 20 KB) | ✓ every visible character identical (exact) | | | **[Agenten-Prompts: Agentenbetrieb (5 Prompts)](agent-prompts-betrieb/onepage.pdf)** | 2,771 words | 20,107 characters at 2.592 mm | 208 KB (original 20 KB) | ✓ every visible character identical (exact) | ## File size, honestly Every glyph carries its own weight, width and size, and in colour mode its own colour; in the PDF every weight × width × type size is a separate embedded font, and every change of style starts a new text run. The picture itself is made of letters only; no image is embedded. Measured on these examples, the one-pager is 0.4 to 2.8 times the size of its PDF source (us-constitution 0.4, un-human-rights 0.7, eu-ai-act 0.9, ch-bundesverfassung 1.6, de-grundgesetz 2.8; below 1 means smaller); plain-text sources grow far more (pep-8: 50 KB → 327 KB; code-review-agent: 3 KB → 105 KB; support-assistant: 3 KB → 84 KB; research-assistant: 3 KB → 67 KB; agents-md: 3 KB → 76 KB; agent-bibliothek: 224 KB → 890 KB; agent-library-readme: 7 KB → 144 KB; agent-library-wissen: 8 KB → 157 KB; agent-regel-quellen: 4 KB → 127 KB; agent-regel-unsicherheit: 4 KB → 119 KB; agent-regel-aufgaben: 4 KB → 122 KB; agent-regel-berechtigungen: 4 KB → 116 KB; agent-regel-datenschutz: 4 KB → 120 KB; agent-regel-qualitaet: 4 KB → 118 KB; agent-regel-ausgabe: 4 KB → 122 KB; agent-regel-fremdinhalte: 4 KB → 115 KB; agent-prompts-code: 19 KB → 207 KB; agent-prompts-sec: 13 KB → 187 KB; agent-prompts-recherche: 19 KB → 200 KB; agent-prompts-dokumente: 18 KB → 201 KB; agent-prompts-daten: 16 KB → 188 KB; agent-prompts-schreiben: 15 KB → 185 KB; agent-prompts-unternehmen: 18 KB → 210 KB; agent-prompts-projekt: 18 KB → 196 KB; agent-prompts-design: 20 KB → 229 KB; agent-prompts-betrieb: 20 KB → 208 KB), because the source is bare text while the one-pager carries fonts and styling. What is done about it, each step measured in [docs/file-size.md](../docs/file-size.md): glyph outlines are rounded to whole units of the 1000-unit glyph grid (invisible: at most 1/2000 of the type size), Chrome's empty accessibility tags are left out, the PDF is repacked with qpdf, photo colours are rounded to steps of 24 (of 255 per channel), and one-pagers use one type size and no width variants, which also makes their pictures sharper. With `--weights` (CLI) or the font-cuts slider (app) fewer weights are used, which makes files much smaller still. The table shows the optimised size next to the original. ## Limits - **Short texts make coarse pictures.** The letters are the pixels: 3 500 characters give about 50 lines of 70 letters, too few for a recognisable title or image (see the AI instruction examples). From about 20 000 characters on, titles and motifs read well. - **Motifs need rights.** Only public-domain images and motifs drawn for this project are used; see motifs/. ## How the text is taken - **PDFs** are read with MuPDF exactly as stored, including page numbers, footnotes and words hyphenated at line ends. The one-pager reproduces the original's text layer, not a cleaned-up copy. - **Text files** (Markdown, reStructuredText) are taken as they are. - Whitespace is normalised to single spaces, as in all Typographics Art. ## Licences of the sources | Example | Licence | |---|---| | ch-bundesverfassung | Not protected by copyright (Art. 5 URG, Swiss Copyright Act: laws and official decrees) | | de-grundgesetz | Not protected by copyright (§ 5 UrhG, German Copyright Act: official works) | | us-constitution | Public domain (work of the United States Government) | | eu-ai-act | Reuse permitted with acknowledgement (Commission Decision 2011/833/EU). Source: EUR-Lex, © European Union | | un-human-rights | United Nations text, freely reproducible | | pep-8 | Public domain (stated in the document) | | code-review-agent | MIT, Typographics Art contributors | | support-assistant | MIT, Typographics Art contributors | | research-assistant | MIT, Typographics Art contributors | | agents-md | MIT, Typographics Art contributors | | agent-bibliothek | Written for this repository (MIT) | | agent-library-readme | Written for this repository (MIT) | | agent-library-wissen | Written for this repository (MIT) | | agent-regel-quellen | Written for this repository (MIT) | | agent-regel-unsicherheit | Written for this repository (MIT) | | agent-regel-aufgaben | Written for this repository (MIT) | | agent-regel-berechtigungen | Written for this repository (MIT) | | agent-regel-datenschutz | Written for this repository (MIT) | | agent-regel-qualitaet | Written for this repository (MIT) | | agent-regel-ausgabe | Written for this repository (MIT) | | agent-regel-fremdinhalte | Written for this repository (MIT) | | agent-prompts-code | Written for this repository (MIT) | | agent-prompts-sec | Written for this repository (MIT) | | agent-prompts-recherche | Written for this repository (MIT) | | agent-prompts-dokumente | Written for this repository (MIT) | | agent-prompts-daten | Written for this repository (MIT) | | agent-prompts-schreiben | Written for this repository (MIT) | | agent-prompts-unternehmen | Written for this repository (MIT) | | agent-prompts-projekt | Written for this repository (MIT) | | agent-prompts-design | Written for this repository (MIT) | | agent-prompts-betrieb | Written for this repository (MIT) | ## Why AI instructions? Instructions for AI systems (system prompts, policies, agent rules such as [AGENTS.md](../AGENTS.md)) are long texts that people rarely read in full. As a one-pager, a person recognises at a glance which instructions a page carries, and a model still receives every rule. That only works if it stays honest: the visible title must name the document, and the full text must be the declared document. Tiny text must never be used to hide instructions from the people who see the page. ## Make your own ```sh node cli/onepage.mjs document.pdf --title "Short title" -o onepage.pdf --png preview.jpg ``` Short titles work best: the fewer characters a document has, the coarser the picture its letters can form.